

ABSTRACT

This study proposes an innovative system based on IoT technology that can effectively monitor the environmental factors affecting astronomical observations. Visibility of astronomical objects including stars, planets, and even meteor showers depends on atmospheric variables such as temperature, humidity, air pressure, amount of dust particles, air pollution, and light pollution. While the traditional method relies on manually estimating those variables or employing complicated and costly equipment at an astronomical observatory, this project proposes a relatively cheaper and portable alternative by harnessing Raspberry Pi along with a variety of sensors.

Raspberry Pi will act as a central processor in this project by collecting and analyzing the input coming from BME280 (for measuring temperature, humidity, and atmospheric pressure), BH1750 (for detecting light intensity), MQ135 (air pollution sensor), GP2Y1010 (dust particle detector), and LDR (light meter). Due to lack of analog input in Raspberry Pi board, this design uses MCP3008 (Analog to Digital Converter) to digitize data coming from MQ135, dust particle detector, and LDR. On the other hand, digital sensors and the OLED display will communicate with Raspberry Pi board via I2C interface.

INDEX

ABSTRACT	ii
CHAPTER 1: INTRODUCTION.....	1
1.1 PURPOSE	2
1.2 PROJECT SCOPE	3
1.3 PRODUCT FEATURES	4
CHAPTER 2: LITERATURE REVIEW	6
CHAPTER 3: SYSTEM ANALYSIS	8
3.1 USER REQUIREMENTS	8
3.1.1 End User Needs	8
3.1.2 Functional Expectations	8
3.1.3 Non-Functional Expectations	9
3.2 HARDWARE REQUIREMENTS	10
3.2.1 Minimum Hardware Requirements	10
3.2.2 Recommended Hardware Requirements	11
3.2.3 Operating System Support	12
3.2.4 User Interface Software Development Technology Stack	12
CHAPTER 4: SYSTEM DESIGN & SPECIFICATIONS	13
4.1 HIGH-LEVEL DESIGN (HLD)	13
4.2 ARCHITECTURE OF THE PROJECT	14
4.3 FLOW CHART	17
4.4 ENTITY RELATION DIAGRAM (ER DIAGRAM)	18
4.5 LOW-LEVEL DESIGN (LLD)	19
CHAPTER 5: CODING.....	23
CHAPTER 6: TESTING	31
6.1 UNIT TESTING	31
CHAPTER 7: CONCLUSION & FUTURE SCOPE	34
7.1 LIMITATIONS	36
CHAPTER 8: REFERENCES	38

CHAPTER 1: INTRODUCTION

Astronomy is one of the sciences. It needs sky to observe celestial bodies. The quality of what astronomers see is affected by things like humidity and temperature and air pressure and dust particles and pollution levels and light intensity. These things can make it hard to see stars and planets and other celestial objects.

In the way astronomers use big telescopes and special systems that have good sensors and satellite data.. These systems are expensive and not many people can use them. Students and small schools usually just look at the sky. Check the weather forecast.. This does not give them good information about the sky.

The DRISHTI project wants to fix this problem. It is making a system that can check the environment all the time and tell us if the sky is good for astronomy. The system uses sensors and a Raspberry Pi to control them. Each sensor checks something like humidity or temperature. The Raspberry Pi looks at all the data. Decides if the sky is good.

The system uses the internet to talk to all the sensors. This makes it easy to use kinds of sensors. The results are shown on a screen and on a website. This makes the system easy to use.

The DRISHTI project shows that we can use technology to solve problems. It also shows that we can use different things together to make something new and useful. Astronomy and the DRISHTI project and the Raspberry Pi and the sensors all work together to help us learn more, about the sky and the celestial bodies.

1.1 PURPOSE

The DRISHTI project wants to make a system that can tell us if the sky is clear and stable.

Astronomical observations need weather.. Many places do not have the right tools to check the weather. They just look at the forecast or guess. This is not very accurate. The DRISHTI project wants to make a system that can check the weather all the time. The DRISHTI project will use a Raspberry Pi computer and many sensors like the BME280 BH1750, MQ135, dust sensor and LDR.

These sensors will give the DRISHTI system the information it needs to figure out if the sky is clear. The DRISHTI system can also use sensors because it has an Analog-to-Digital Converter. This makes the DRISHTI system very flexible.

The DRISHTI project also wants to show people how to use the internet and special computers to solve problems. The project will teach people about sensors. How to get information from them in real time. This will help students learn how to make systems that can do things.

The DRISHTI project wants to be cheap and easy to use. It wants to help schools and people who like to study the stars. The DRISHTI project uses sensors and open-source computers. This makes the DRISHTI system affordable and good, at what it does.

In short the DRISHTI project is making a system to monitor the environment. Predict if the sky is clear. The DRISHTI project is also making a system that can teach people and be used by many. The DRISHTI project is using sensors, special computers and the internet to make this happen. The DRISHTI project is a way to learn about the environment and the stars.

1.2 PROJECT SCOPE

- The DRISHTI project is about designing and building a system that uses the internet of things to monitor the environment. This system is meant to help people who like to look at the stars like students, researchers and people who do it for fun. The DRISHTI project is supposed to be a way to monitor the environment in real time so people can use it in schools, labs and at home.
- The DRISHTI project mainly involves connecting sensors to a small computer called a Raspberry Pi. These sensors measure things like temperature, humidity and air pollution which're important for seeing the stars clearly at night. By collecting data from these sensors all the time the system helps us understand what is going on in the environment around us.
- The DRISHTI project also involves collecting and processing data in time using the Raspberry Pi. The system is designed to read data from the sensors process it and show us what it means for seeing the stars. The results are shown on a screen and can also be seen from far away using a web page, which makes it easy to use.
- However the DRISHTI project is limited to monitoring the environment from the ground. Does not involve using satellites or expensive equipment for astronomy. The focus is on making a affordable and educational system that shows how the internet of things and embedded hardware can be used in a practical way.
- In conclusion the DRISHTI project is about putting hardware connecting sensors monitoring the environment in real time and processing data which makes it a complete and scalable solution, for analyzing the visibility of the sky and monitoring the environment. The DRISHTI project is a way to learn about the environment and how to use technology to make our lives better.

1.3 PRODUCT FEATURES

- The DRISHTI system is really good at monitoring the environment for observations. It has a lot of features that make it very useful.
- The DRISHTI system also uses a lot of sensors. It has analog sensors that work together to measure all sorts of environmental things. This makes the data more accurate and reliable because it is looking at a lot of factors.
- The DRISHTI system can communicate in two ways using I2C and SPI protocols. Some sensors, like the BME280 and BH1750 use the I2C protocol while others, like the MQ135 and dust sensor use the SPI protocol. This shows that the DRISHTI system is very flexible and can work with a lot of types of sensors.
- This means that users can see the sky visibility status and sensor readings without needing to use a device. This makes the DRISHTI system very portable and easy to use.
- The DRISHTI system also allows for monitoring using a web dashboard. This means that users can see the data from anywhere as long as they have a device connected to the network. This makes the DRISHTI system very useful because users can monitor the environment from anywhere.
- The DRISHTI system is also very affordable and compact. It uses sensors and a Raspberry Pi, which makes it very cost-effective. The hardware is also very small which makes it easy to install and use in places.
- The DRISHTI system is also very flexible. Can be easily modified. New sensors or modules can be added to the system without needing to make a lot of changes. This makes the DRISHTI system very good for projects because it can be easily updated and improved.
- The DRISHTI system is also energy efficient and reliable. It is designed to use little power while still working very well. The system is also very stable because it has grounding and power management.
- The DRISHTI system is an example of how IoT and embedded systems can be used in a practical way. It shows how different hardware components can be

used together to do something which makes it a great tool for learning.

- In summary the DRISHTI system is very good at monitoring the environment and analyzing sky visibility. It has a lot of features including real-time monitoring, multi-sensor integration and remote data display which make it a robust and effective solution. The DRISHTI system is a tool, for anyone who needs to monitor the environment and it is a great example of what can be done with IoT and embedded systems.

CHAPTER 2: LITERATURE REVIEW

1. The Internet of Things (IoT) and embedded systems have changed environmental monitoring systems a lot in years. Many researchers and developers are working on systems that measure temperature, humidity, air pressure, pollution and light intensity. These systems are used in weather forecasting, environmental analysis, smart agriculture and industrial monitoring.
2. Traditional weather monitoring systems use satellite data and big infrastructure. They provide global predictions but can't give precise local information. This is a problem in observation, where small changes in local conditions affect visibility. So there's a growing interest in localized ground-based monitoring systems using low-cost hardware.
3. Some studies use microcontrollers like Arduino and single-board computers like Raspberry Pi for monitoring. Arduino-based systems are popular because they're simple and easy to use with sensors.. They can't handle complex data processing or advanced applications.
4. Researchers have also looked into using sensors to improve environmental monitoring systems. Multi-sensor systems give a picture of atmospheric conditions by measuring different parameters at the same time. For example temperature and humidity sensors help understand weather conditions while gas sensors and dust sensors provide information about air quality.
5. In astronomy predicting sky visibility is usually done using telescopes, observatories and satellite imagery.
6. Machine learning techniques can also be used with IoT systems. Machine learning algorithms can analyze data and identify patterns to predict future conditions. In monitoring machine learning can predict weather conditions, air quality and other parameters.
7. Analog-to-Digital Converters (ADCs) are also important in embedded systems. Many sensors provide analog outputs so ADCs are needed to convert these signals into form for processing.

8. The DRISHTI project combines hardware-based sensor integration with real-time data acquisition and basic predictive analysis. It integrates analog sensors using I2C and SPI protocols showing a comprehensive approach to embedded system design. The project aims to provide a cost and accessible solution for educational environments. It uses components and open-source platforms making it easy to replicate and modify.
9. The DRISHTI project addresses the need for systems that combine affordability, simplicity and effective hardware integration. It provides an scalable solution, for monitoring sky visibility conditions using a combination of sensors, embedded systems and IoT technologies.

CHAPTER 3: SYSTEM ANALYSIS

3.1 USER REQUIREMENTS

3.1.1 End User Needs

The People who like to look at the stars need a system that can tell them what the environment is like now. This system has to be good at giving them information so they can see if the sky is good for looking at stars, planets and meteor showers. The system also has to be easy to use. They might want to use it on a roof in a field or in a laboratory. So the system has to be small not heavy and easy to set up. The system also has to be cheap. It is for students and schools so it has to be made with parts that do not cost a lot of money. The astronomical observation system has to be affordable without not working. The astronomical observation system is very important, for people who like to look at the stars. It has to meet all these needs.

3.1.2 Functional Expectations

The DRISHTI system is supposed to do things. The DRISHTI system is expected to do important things to meet what the users need. The DRISHTI system should be able to get real time information from sensors. These sensors include temperature and humidity and pressure and light intensity and dust concentration and air pollution levels. The DRISHTI system has to get this information all the time. It has to be correct. Another important thing the DRISHTI system should do is process the information it gets from the sensors. The DRISHTI system should take the information it gets from the sensors. Make it into something that makes sense. This can include calculating a score that shows how visible the sky is or figuring out if the sky is good for observing. The DRISHTI system should also show the results in ways. It has to show the results, on a screen so people can see them locally and on a web dashboard so people can see them from far away.

3.1.3 Non-Functional Expectations

The system should work all the time without breaking down a lot and give us the results every time. Reliability is what makes the system good. The system also needs to perform. It should be able to handle information and give us updates right away without taking too long. Performance is a part of the system. We also need to think about how energy the system uses. The system should not use much power and should still work well especially if it is going to be used for a long time. Energy efficiency is important for the system. It is also important that the system is easy to fix if something goes wrong. The system should be easy to figure out what is wrong with it and fix it whether it is a problem, with the hardware or the software. Maintainability is a part of the system.

3.2 HARDWARE REQUIREMENTS

3.2.1 Minimum Hardware Requirements

- The minimum hardware requirements include the parts needed to build a basic version of the DRISHTI system. These parts help the system do tasks like collecting data processing it and showing it on a screen.
- The Raspberry Pi is the computer and controls the whole system. A basic Raspberry Pi 3 or Raspberry Pi 4 is enough to handle sensor data and simple tasks.
- The system needs sensors like BME280 and BH1750. These measure temperature, humidity, air pressure and light. They are digital. Talk to the Raspberry Pi using I2C.
- The system also uses analog sensors, like MQ135 gas sensor, dust sensor and LDR. They measure air quality, dust and light. The Raspberry Pi can't read analog signals so it needs an MCP3008 converter to change them into signals using SPI.
- An OLED display shows sensor readings and sky visibility. It is a way to see data locally.
- Other needed parts include a breadboard, jumper wires, resistors and a reliable power supply. These help connect everything properly. Make sure the system works well.

3.2.2 Recommended Hardware Requirements

- **Raspberry Pi 4 Model B** – Acts as the central processing unit responsible for data acquisition, processing, and system control.
- **BME280 Sensor** – Measures temperature, humidity, and atmospheric pressure for environmental analysis.
- **BH1750 Light Sensor** – Detects ambient light intensity to analyze light pollution and sky brightness.
- **MQ135 Gas Sensor** – Monitors air quality and pollution levels affecting sky visibility.
- **GP2Y1010AU0F Dust Sensor** – Measures particulate matter concentration in air for visibility estimation.
- **LDR (Light Dependent Resistor)** – Detects overall light levels and helps identify day/night conditions.
- **10 k Ω Resistor** – Used with LDR to form a voltage divider for accurate analog readings.
- **MCP3008 ADC Module** – Converts analog signals from sensors into digital data for Raspberry Pi processing.
- **0.96-inch OLED Display (I2C)** – Displays real-time sensor values and sky visibility status.
- **Breadboard** – Provides a platform for assembling and testing the circuit without soldering.
- **Jumper Wires (Male-to-Male / Male-to-Female)** – Used to connect all hardware components efficiently.
- **Power Adapter (5V, 3A)** – Supplies stable power to the Raspberry Pi and connected components.
- **Protective Enclosure (Optional)** – Protects the system from environmental conditions during outdoor deployment.

3.2.3 Operating System Support

The DRISHTI system is made to work with Linux based systems that are compatible with the Raspberry Pi platform. I like to use Raspberry Pi OS because it works well is very stable and supports things like I2C and SPI which are necessary for the DRISHTI system to talk to sensors. The DRISHTI system also works with Linux systems like Ubuntu and Debian. These systems give us a lot of flexibility can do things at the same time and support Python, which is what we use for the DRISHTI systems internet connected applications. These systems have a lot of tools and libraries that the DRISHTI system needs to work with sensors, process data and make web pages. We can also use Windows IoT Core if we want to. It is not as popular for the DRISHTI systems kind of projects. The DRISHTI system likes these systems because they have a terminal that makes it easy to set up debug and manage the DRISHTI system, which's very important, for the DRISHTI project.

3.2.4 User Interface Software Development Technology Stack

- The DRISHTI systems user interface is built using web technologies. These technologies help show sensor data in time in an easy-to-use way.
- The frontend is made with HTML and CSS.
- HTML creates the layout of the web dashboard.
- CSS makes it look good with colors and designs that work on devices.
- JavaScript is used to handle changing data and make the interface interactive.
- It updates sensor values. Shows changes without needing to reload the page.
- The backend is built using the Flask framework with Python.
- Flask connects the hardware and the user interface.
- This helps the hardware and interface communicate smoothly.
- Chart.js can be used to show sensor data, in charts and graphs.
- This makes it easy for users to understand trends.
- The technology used is lightweight, efficient and easy to set up.
- This helps users monitor sky visibility conditions easily.

CHAPTER 4:

SYSTEM DESIGN AND SPECIFICATION

The DRISHTI project system design phase is an important part of the DRISHTI project development. This is where we figure out the layout and how all the parts of the DRISHTI project system work together. We need to make sure the hardware and software parts of the DRISHTI project system are working well together so that the DRISHTI project system runs smoothly and does what it is supposed to do. The DRISHTI project system design is split into two parts: the High-Level Design and the Low-Level Design of the DRISHTI project system. The High-Level Design of the DRISHTI project system gives us a picture of how the DRISHTI project system is laid out. This includes how the sensors and the processing unit and the output devices all interact with each other. It is about how all the different parts of the DRISHTI project system talk to each other and how the data moves around the DRISHTI project system. This helps us understand the DRISHTI project system as a whole without getting too caught up in the details. The Low-Level Design of the DRISHTI project system is more about the details of how the DRISHTI project system works. This includes things like the algorithms and the step-, by-step processes and how each part of the DRISHTI project system works internally. This gives us an understanding of how the DRISHTI project system operates when it is running.

4.1 HIGH-LEVEL DESIGN (HLD)

The DRISHTI system is made up of a design that shows how all the parts work together. This design tells us how the different hardware parts are connected and how the data gets from the sensors to the parts that show the output. The DRISHTI system is made in a way that has three parts: the part that gets the data the part that does the processing and the part that shows the output.

The part that gets the data has sensors that check the environment all the

time. These sensors measure things like the temperature how humid it is, the pressure, how bright it's how much dust is in the air and how polluted the air is. These sensors are connected to the Raspberry Pi using ways of talking to each other like I2C and SPI. The digital sensors can talk to the Raspberry Pi directly using I2C and the sensors that are not use the MCP3008 ADC and talk using SPI.

The part that does the processing is the Raspberry Pi, which's like the brain of the DRISHTI system. It gets all the data from the sensors does something with it and then makes sense of it so we can see how clear the sky is. The Raspberry Pi does things like reading what the sensors say changing the signals, from the sensors that're not digital and getting the data ready to show.

The part that shows the output has an OLED display and a web dashboard. The OLED display shows what is happening in time and the web dashboard lets us check on things from far away using a browser. This way of making the DRISHTI system makes sure that the data flows well and that each part can be worked on separately. The DRISHTI system is an example of a system that is made to be easy to understand and work with. The DRISHTI system has parts that work together to make it useful.

4.2 ARCHITECTURE OF PROJECT

The DRISHTI system is made up of layers. Each layer of the DRISHTI system does something. These layers work together to make the DRISHTI system work properly. This makes the DRISHTI system easy to understand. It is also easy to put the DRISHTI system in place and keep it running smoothly. The DRISHTI system is designed to be flexible and handle a lot of data. The layers of the DRISHTI system help, with this. They make sure the DRISHTI system can grow and change as needed.

- **Data Acquisition Layer (Sensor Layer):-**

The Data Acquisition Layer is the layer of the system and it is very important. It is responsible for collecting data in real time. This layer has sensors that measure different things in the air that affect how clear the sky is. The BME280 sensor is used to measure temperature and humidity and pressure. These things are necessary to understand the weather and how clouds form. The BH1750 sensor measures how bright it is outside which helps us understand how light pollution there is and how bright the sky is. The MQ135 sensor is present to detect how polluted the air is. The LDR or Light Dependent Resistor is used to learn how light it is outside and to tell the difference between day and night. These sensors are always generating data, which is sent to the processing layer. The digital sensors use the I2C protocol to communicate. The analog sensors send signals to the MCP3008 ADC so it can be converted into something that can be used. The Data Acquisition Layer and its sensors are very important, for collecting Data Acquisition Layer information.

- **Signal Conversion Layer (ADC Layer):-**

The Raspberry Pi does not support analog input so here we use the MCP3008 Analog-to-Digital Converter in this layer for betterment of the project. The MCP3008 gets the analog signals from sensors, like the MQ135, the dust sensor and the LDR and use it for further process. It then changes these signals into values using SPI communication. This helps the Raspberry Pi to understand the sensor data correctly. The Signal Conversion Layer is used to make sure that the analog sensors work well with the processing system. The Signal Conversion Layer is a part of this because it helps the Raspberry Pi understand what the analog sensors are saying.

- **Processing Layer (Controller Layer):-**

The Processing Layer is the part of the system where everything gets done. This is where the Raspberry Pi comes in it is like the brain of the thing. The

Raspberry Pi gets information from two kinds of sensors that are present in the project: sensors and analog sensors. It uses I2C to talk to the sensors. Then it figures out what is going on with the environment and if the sky is visible. The Raspberry Pi does a lot of work to make sense of all the information it gets like cleaning it up and putting it in order. The Raspberry Pi is also in charge of making sure all the parts of the system work together smoothly. The Processing Layer is really important because it takes all the information, from the sensors and turns it into something that actually means something. The Raspberry Pi is what makes the system work it is the controller and it does a great job of managing everything.

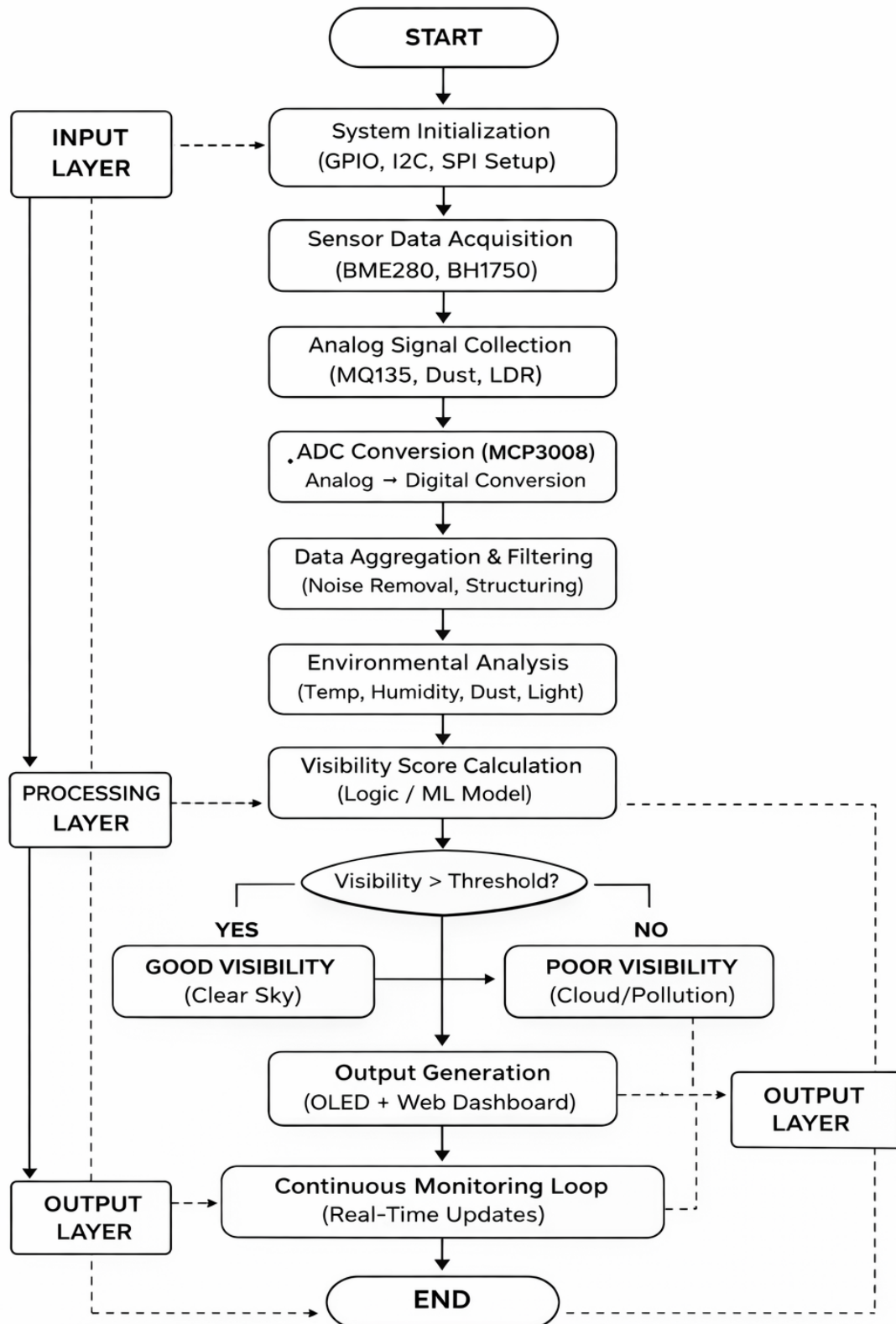
- **Application Layer (Software & Analysis Layer):-**

The Application Layer is responsible for analyzing the processed data and generating output in a user-understandable format so that it will be very much helpful. This layer includes software components such as Python scripts, machine learning models, and web frameworks. The system may use basic machine learning algorithms to calculate a sky visibility score based on sensor inputs. It also prepares data for visualization and user interaction. This layer acts as a bridge between hardware processing and user interface.

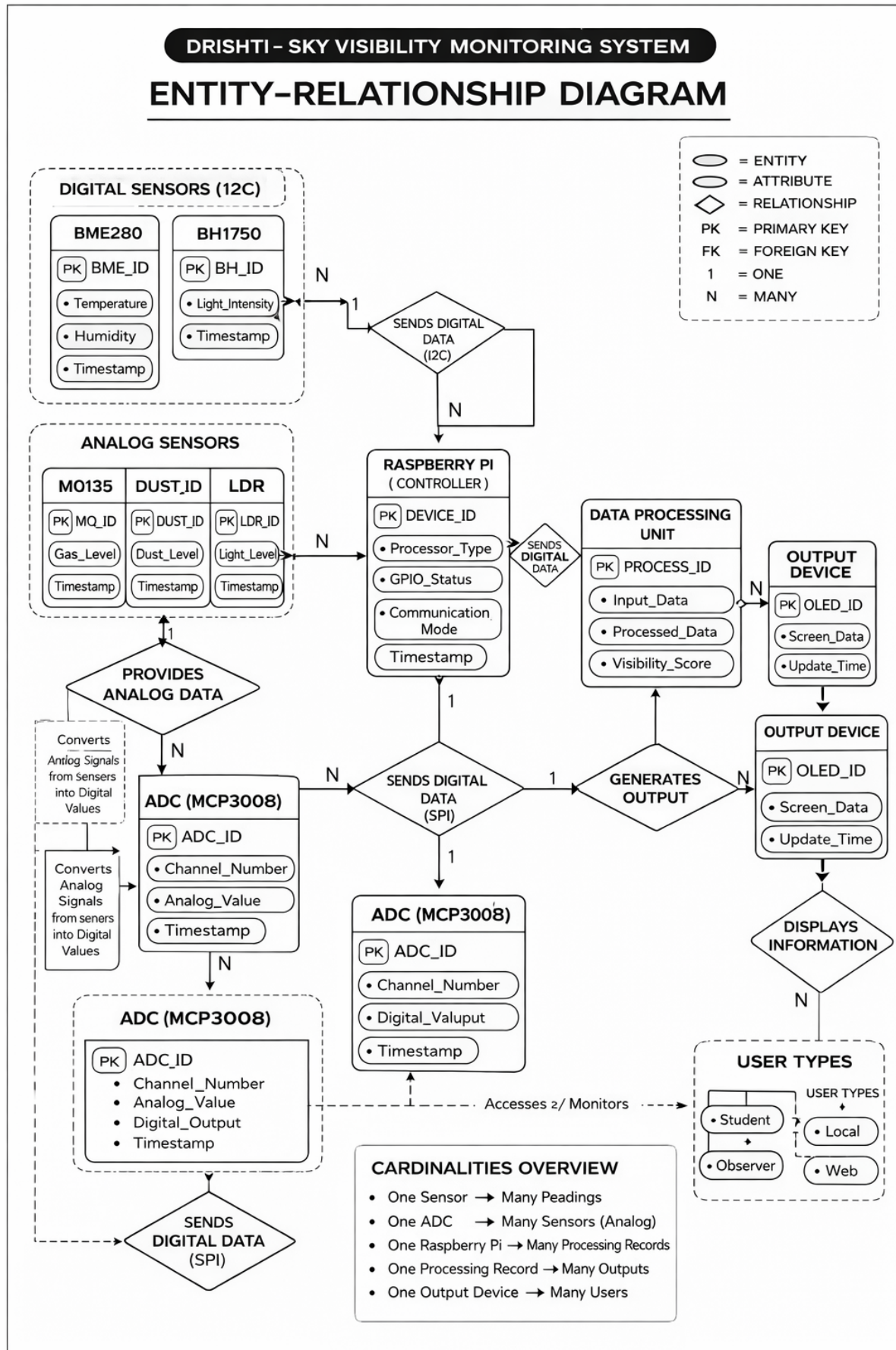
- **Output Layer (Display & Interface Layer):-**

The Output Layer is the part of the system. It shows the results to the user. The Output Layer has the OLED display and the web dashboard. The OLED display gives you information away like what the sensors are reading and if you can see the sky. The web dashboard lets you check the data, from else using a browser. You can see pictures and graphs which makes it easier to understand. The Output Layer makes sure that the information that is provided it gives you is easy to read and understand. The Output Layer delivers the information in a way that's clear and easy to use which is a great thing.

4.3 FLOW CHART



4.4 ENTITY RELATION DIAGRAM (E-R DIAGRAM)



4.5 LOW-LEVEL DIAGRAM (LLD)

The Low-Level Design (LLD) of the DRISHTI system focuses on the detailed internal working of each module involved in the system. It explains how data is collected from sensors, processed, filtered, and finally converted into meaningful output such as sky visibility status. Unlike High-Level Design, which provides an overview, the LLD describes the step-by-step functioning of the system at a component level. The LLD ensures that each hardware component and software module works in synchronization to achieve real-time monitoring and accurate analysis. It includes sensor data acquisition, signal processing, filtering, and calculation of sky visibility conditions based on environmental parameters.

4.5.1 Reading Sensor Data Inside Room

The first step in the Low-Level Design is getting data from sensors placed in the environment. The DRISHTI system uses sensors to collect real-time data about the atmosphere that affects how clear the sky is.

These sensors include:

- * BME280 for temperature, humidity and pressure
- * BH1750 for intensity
- * MQ135 for air pollution
- * GP2Y1010 dust sensor for particulate matter
- * LDR for general light detection

The Raspberry Pi is like the main controller. It reads data from both analog sensors. Digital sensors like BME280 and BH1750 present here use the I2C communication protocol. This protocol lets multiple devices share the communication lines. The Raspberry Pi regularly sends requests to these sensors. Gets data in digital format. For sensors like MQ135, dust sensor and LDR the system uses the MCP3008 Analog-to-Digital Converter. These sensors produce analog voltage signals. These signals go to the MCP3008 first. The ADC converts these analog signals into values using SPI communication. This lets the Raspberry Pi

read. Process the data. The data collection process is ongoing. Happens at regular intervals. This ensures real-time monitoring of conditions. Each sensor reading has a timestamp. This helps track changes over time and analyze trends and give a good result. The DRISHTI system and Raspberry Pi present here work together to collect and process data from sensors like BME280, BH1750, MQ135, GP2Y1010 and LDR. The data, from these sensors helps understand the environment.

4.5.2 Processing & Filtering Sensor Signals

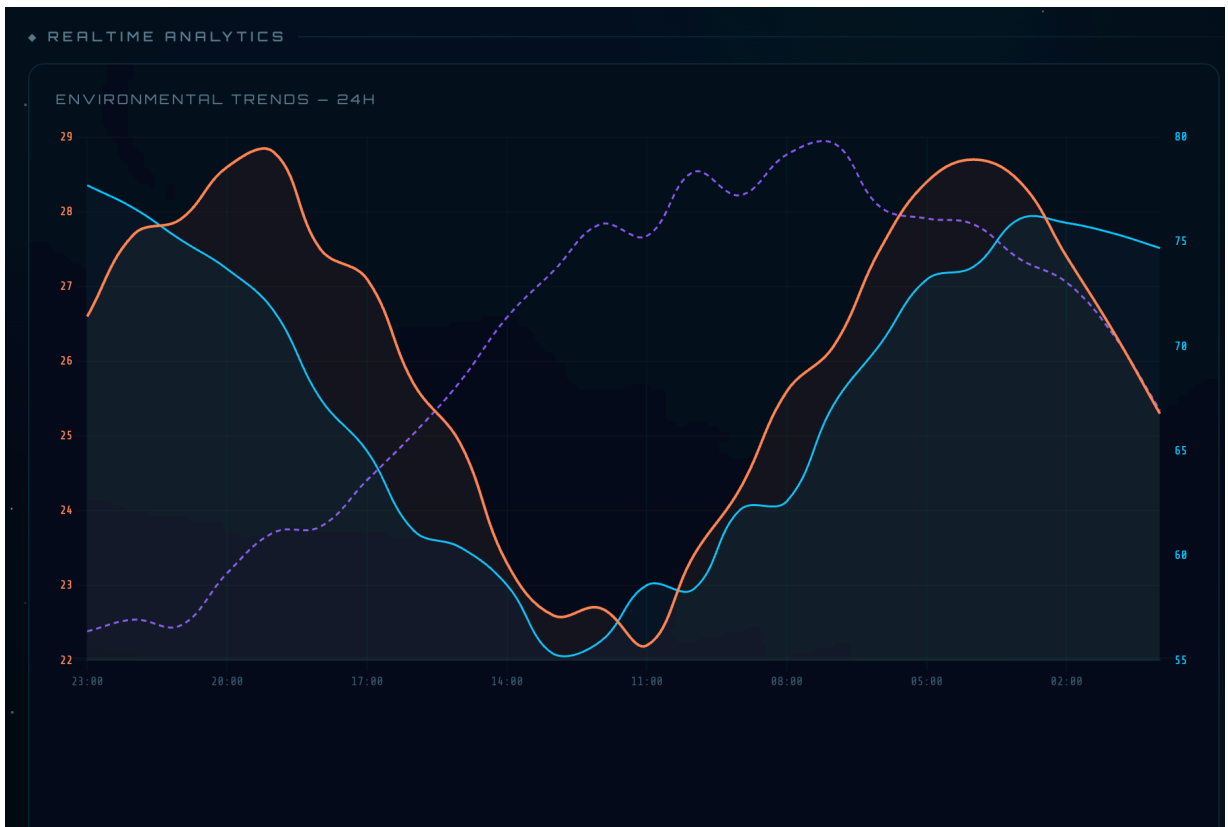
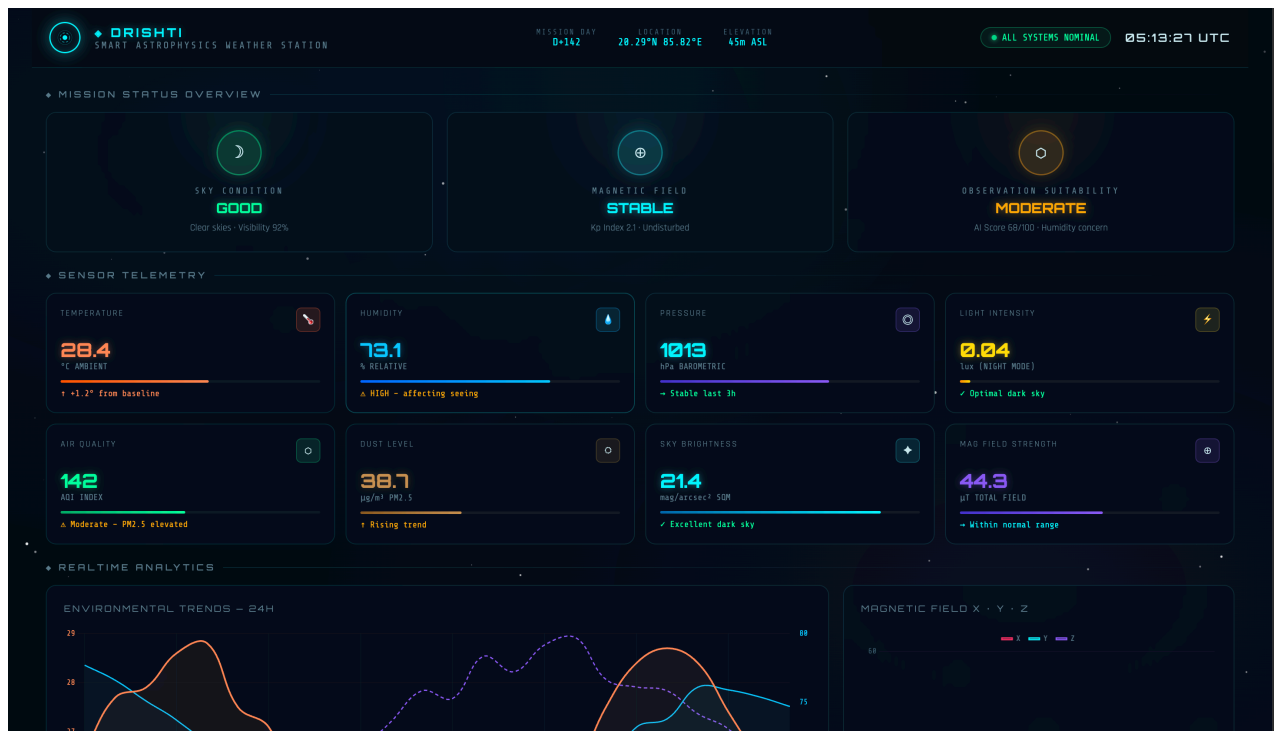
When we get the sensor data the next thing we do is process and filter the data to make sure it is accurate and reliable. The raw sensor data can have mistakes. Go up and down because of things around it or because the sensor is not perfect. So we need to filter the data before we look at it closely. The Raspberry Pi takes the collected data. Puts it in a neat order. If we find any wrong values we ignore them or correct them so they do not cause mistakes in the final result. This makes the system more reliable. Gives us accurate answers. After we filter the data we store it for a while and get it ready for more analysis. We might also make all the sensor values similar so they are on the scale, which makes it easier to compare and look at many things at the same time. This part is very important to make the system accurate because it makes sure we only use good and meaningful sensor data to figure out how visible the sky is. We use sensor data to calculate sky visibility. We want to make sure the sensor data is clean and good. The sensor data is used to give us a picture of the sky and that is why this stage is so important, for calculating sky visibility with sensor data.

4.5.3 Sky Visibility Calculation

The last part of the Low-Level Design is to figure out how visible the sky is by using the data that has been processed. This step takes the readings from the sensors. Turns them into information that the user can understand easily. The system looks at a lot of things like how humid it's how hot or cold it is, how much dust is in the air how polluted it is and how bright it is to decide how good the sky

visibility is. But if it is not very polluted and the light is just right the sky will be clear. The DRISHTI system gives a score for how visible the sky's based on some rules or a simple machine learning model. This score can be low or high which means the sky can be hard to see or easy to see. There are numbers that are used to say if the sky is visible or not like "Good Visibility" or "Poor Visibility". If the score is higher than a number the system says it is a good time to look at the stars. If it is not higher than that number it says the sky is not good for looking at the stars. The system uses some logic to make this decision. Then the system sends the answer to the OLED display and the web dashboard so the user can see it away. The system keeps updating what it says about the sky by getting new data and processing it. This part of the DRISHTI system is the important part because it takes the raw data from the sensors and turns it into something the user can use. The DRISHTI system is about helping the user and this part is what makes that happen. The DRISHTI system is the key to getting information, about the sky and this module is the core of the DRISHTI system.

4.6 SCREENSHOT



CHAPTER 5: CODING

```
import time

import spidev

import board

import busio

import smbus

import json

import threading

import statistics

import adafruit_bme280


class SensorManager:

    def __init__(self):

        self.i2c = busio.I2C(board.SCL, board.SDA)

        self.bme280 = adafruit_bme280.Adafruit_BME280_I2C(self.i2c)

        self.bus = smbus.SMBus(1)

        self.spi = spidev.SpiDev()

        self.spi.open(0, 0)

        self.spi.max_speed_hz = 1350000

        self.light_addr = 0x23


    def read_bme(self):

        return {

            "temperature": self.bme280.temperature,

            "humidity": self.bme280.humidity,
```

```

        "pressure": self.bme280.pressure
    }

```

```

def read_light(self):
    data = self.bus.read_i2c_block_data(self.light_addr, 0x20)
    return (data[0] << 8 | data[1]) / 1.2

```

```

def read_adc(self, ch):
    adc = self.spi.xfer2([1, (8 + ch) << 4, 0])
    return ((adc[1] & 3) << 8) + adc[2]

```

```

def read_all(self):
    data = self.read_bme()
    data["light"] = self.read_light()
    data["gas"] = self.read_adc(0)
    data["dust"] = self.read_adc(1)
    data["ldr"] = self.read_adc(2)
    data["timestamp"] = time.time()
    return data

```

```

class DataFilter:
    def __init__(self):
        self.history = []

    def smooth(self, data):

```

```

self.history.append(data)

if len(self.history) > 5:
    self.history.pop(0)

avg = {}

for key in data:
    if key != "timestamp":
        avg[key] = statistics.mean([d[key] for d in self.history])

avg["timestamp"] = data["timestamp"]

return avg

```

```

class VisibilityEngine:

    def __init__(self):
        self.weights = {
            "humidity": 0.25,
            "gas": 0.20,
            "dust": 0.20,
            "light": 0.20,
            "ldr": 0.15
        }

    def normalize(self, value, max_val):
        return min(value / max_val, 1)

    def compute_score(self, data):
        h = self.normalize(data["humidity"], 100)

```

```

g = self.normalize(data["gas"], 800)
d = self.normalize(data["dust"], 600)
l = self.normalize(data["light"], 100)
r = self.normalize(data["ldr"], 1023)

score = 100 - (
    h * self.weights["humidity"] * 100 +
    g * self.weights["gas"] * 100 +
    d * self.weights["dust"] * 100 +
    l * self.weights["light"] * 100 +
    r * self.weights["ldr"] * 100
)

if score < 0:
    score = 0

return score

def classify(self, score):
    if score >= 75:
        return "EXCELLENT"
    elif score >= 60:
        return "GOOD"
    elif score >= 40:
        return "MODERATE"
    else:

```

```
return "POOR"
```

```
class DataLogger:
```

```
    def __init__(self, file="drishti_log.json"):
```

```
        self.file = file
```

```
    def log(self, data):
```

```
        try:
```

```
            with open(self.file, "a") as f:
```

```
                f.write(json.dumps(data) + "\n")
```

```
        except:
```

```
            pass
```

```
class SystemController:
```

```
    def __init__(self):
```

```
        self.sensor = SensorManager()
```

```
        self.filter = DataFilter()
```

```
        self.engine = VisibilityEngine()
```

```
        self.logger = DataLogger()
```

```
        self.running = True
```

```
    def process_cycle(self):
```

```
        raw = self.sensor.read_all()
```

```
        filtered = self.filter.smooth(raw)
```

```

score = self.engine.compute_score(filtered)
status = self.engine.classify(score)

result = {
    "temperature": round(filtered["temperature"], 2),
    "humidity": round(filtered["humidity"], 2),
    "pressure": round(filtered["pressure"], 2),
    "light": round(filtered["light"], 2),
    "gas": int(filtered["gas"]),
    "dust": int(filtered["dust"]),
    "ldr": int(filtered["ldr"]),
    "visibility_score": round(score, 2),
    "status": status,
    "timestamp": filtered["timestamp"]
}

self.logger.log(result)

return result

```

```

def display(self, data):
    print("\nDRISHTI SYSTEM")
    for k, v in data.items():
        if k != "timestamp":
            print(k.upper(), ":", v)

```

```

def run(self):

```

```
while self.running:
    try:
        data = self.process_cycle()
        self.display(data)
        time.sleep(2)
    except:
        time.sleep(2)
```

```
class BackgroundService:
    def __init__(self, controller):
        self.controller = controller

    def start(self):
        t = threading.Thread(target=self.controller.run)
        t.daemon = True
        t.start()
```

```
def main():
    controller = SystemController()
    service = BackgroundService(controller)
    service.start()

    try:
        while True:
```

```
        time.sleep(1)
    except KeyboardInterrupt:
        controller.running = False

if __name__ == "__main__":
    main()
```

CHAPTER 6: TESTING

6.1 UNIT TESTING

We did some unit testing to see if the parts of the DRISHTI system work properly. We wanted to make sure each part of the DRISHTI system is working fine before we put everything together. So we tested each sensor and part of the DRISHTI system on its own to see how well it works how accurate it is and how it responds when things around it change. The following tables show what we did during these tests, for the DRISHTI system.

6.1.1 Test Case Table 1: BME280 Sensor (Temperature, Humidity, Pressure) Testing:-

Test Case ID	Input	Expected Output	Actual Output	Status	Remarks
BME-01	Normal room condition	Temperature, humidity, pressure values displayed	As expected	Pass	Sensor working correctly
BME-02	High humidity environment	Increased humidity value	As expected	Pass	Accurate humidity detection
BME-03	Temperature change (heat source)	Temperature rises gradually	As expected	Pass	Good sensitivity
BME-04	Stable condition	Constant readings	As expected	Pass	Stable output

6.1.2 Test Case Table 2: BH1750 Sensor (Light Intensity) Testing:-

Test Case ID	Input	Expected Output	Actual Output	Status	Remarks
BH-01	Bright light	High lux value	As expected	Pass	Accurate light detection
BH-02	Dim light	Low lux value	As expected	Pass	Works in low light

BH-03	Sudden light change	Immediate variation in output	As expected	Pass	Fast response
BH-04	No light (dark)	Near zero lux value	As expected	Pass	Correct dark detection

6.1.3 Test Case Table 3: MQ135 Gas Sensor (Air Quality) Testing:-

Test Case ID	Input	Expected Output	Actual Output	Status	Remarks
MQ-01	Normal air	Moderate analog value	As expected	Pass	Stable baseline
MQ-02	Smoke exposure	Increased gas value	As expected	Pass	Detects pollution
MQ-03	Clean air	Reduced value	As expected	Pass	Proper detection
MQ-04	Continuous exposure	Gradual increase	As expected	Pass	Responsive behavior

6.1.4 Test Case Table 4: LDR Sensor (Ambient Light) Testing:-

Test Case ID	Input	Expected Output	Actual Output	Status	Remarks
LDR-01	Bright light	High analog value	As expected	Pass	Accurate in bright light
LDR-02	Low light	Low analog value	As expected	Pass	Works in dim light
LDR-03	Covered sensor	Near zero value	As expected	Pass	Detects darkness
LDR-04	Gradual light change	Smooth variation	As expected	Pass	Good response

6.1.5 Test Case Table 5: Overall System (DRISHTI) Testing:-

Test Case ID	Input	Expected Output	Actual Output	Status	Remarks
SYS-01	Clear environment	High visibility score	As expected	Pass	Good performance
SYS-02	High pollution + dust	Low visibility score	As expected	Pass	Accurate prediction
SYS-03	Moderate conditions	Medium score	As expected	Pass	Balanced output
SYS-04	Continuous monitoring	Real-time updates	As expected	Pass	Stable system

CHAPTER 7: CONCLUSION & LIMITATIONS

The DRISHTI project is really good at showing how to design and build a system that uses hardware and the internet to monitor the environment and how it affects what we can see in the sky when we look at the stars. The main goal of this project was to make a system that is cheap works well and can give us information in real time about what is going on in the air and whether we can see the sky clearly. The system uses a lot of sensors and a Raspberry Pi to control everything, which is a great way to learn about how to connect sensors and use the internet and find a good way.

This is really flexible. Means the system can handle a lot of different kinds of information from the sensors. The DRISHTI project uses sensors to get information about the environment and the DRISHTI project is good at collecting this information.

The system is really good at collecting information about the environment like the temperature how humid it is, the pressure, how bright it's the air quality and how much dust is in the air. All of these things are important for figuring out whether we can see the sky clearly. The DRISHTI project uses this information to help us understand what is going on in the environment. The DRISHTI project also uses techniques to make sure the information it collects is accurate and not noisy which makes the system more reliable.

Another important thing about the project is that it can figure out how visible the sky is. The DRISHTI system uses a way of scoring to look at the environment and decide whether the sky is excellent good moderate or poor. The DRISHTI project is good at helping us decide whether it is a time to look at the stars. The decision-making part of the system is simple and easy to use so people can quickly figure out whether the sky is good for looking at the stars.

The DRISHTI system is also good at letting people see what is going on in

time. The information is shown on a screen and it can also be seen on a website from anywhere. This is really useful. Means people can get the information they need easily. The DRISHTI project is always collecting information and updating the system so people can always see what is going on. The DRISHTI system is good at showing us what is happening in the environment.

From a hardware point of view the project is really well done. The sensors are all connected correctly. The communication protocols are used in a good way. The DRISHTI project uses a chip to convert the analog signals to digital signals, which is really important. The circuit is designed to be stable. The use of other components like resistors and a power supply helps make the system reliable. The DRISHTI project is good at using hardware to collect information.

The project is also a way to learn about the internet and embedded systems. It gives people hands-on experience with connecting sensors collecting information and integrating systems. The DRISHTI project uses a Raspberry Pi as the controller, which means it can do both hardware and software things making it really versatile. The DRISHTI project is good at teaching us about the internet and embedded systems.

In terms of cost the DRISHTI system is really good. It uses parts, which makes it accessible to students and schools. This makes it good for both learning and practical uses. The DRISHTI project is good at being affordable and easy to use.

However the project does have some limitations. The system also uses logic to figure out how visible the sky is, which could be improved with more advanced techniques. The DRISHTI project is not perfect. It is still really good at what it does.

7.1 LIMITATIONS

- Not medical-grade
 - The system is designed for educational and experimental purposes only and cannot replace professional observatory equipment or scientific-grade instruments. The readings should not be used for critical astronomical research.
- Limited sensing coverage
 - The system monitors only the immediate surrounding environment where it is installed. Conditions in other locations or wider atmospheric layers may not be accurately represented.
- Dependent on sensor accuracy and calibration
 - The performance of the system heavily depends on the quality and calibration of sensors. Inaccurate or uncalibrated sensors may lead to incorrect visibility predictions.
- Environmental interference
 - External factors such as sudden weather changes, artificial lighting, nearby pollution sources, and physical obstructions may affect sensor readings and reduce system accuracy.
- Limited parameter consideration
 - The system considers only a specific set of environmental parameters such as temperature, humidity, dust, pollution, and light intensity. Other factors like wind speed, cloud type, and atmospheric turbulence are not included.
- Basic visibility calculation model
 - The current system uses a simple weighted logic for calculating sky visibility. It does not implement advanced machine learning models or predictive algorithms for higher accuracy.

- No long-term prediction capability
 - The system focuses on real-time monitoring and does not analyze historical data to predict future sky conditions or trends.
- Power dependency
 - The system requires a continuous power supply for operation. Any power interruption may stop monitoring and affect data continuity.

CHAPTER 8.

REFERENCES

- Raspberry Pi Foundation, “*Raspberry Pi Documentation*”, Available: <https://www.raspberrypi.org/documentation/>
- Bosch Sensortec, “*BME280 Environmental Sensor Datasheet*”, 2020.
- Rohm Semiconductor, “*BH1750 Digital Light Sensor Datasheet*”, 2011.
- Hanwei Electronics, “*MQ135 Gas Sensor Datasheet*”, 2015.
- Sharp Corporation, “*GP2Y1010AU0F Dust Sensor Datasheet*”, 2006.
- Microchip Technology Inc., “*MCP3008 Analog-to-Digital Converter Datasheet*”, 2016.
- Adafruit Industries, “*Interfacing Sensors with Raspberry Pi*”, Available: <https://learn.adafruit.com>
- Python Software Foundation, “*Python Documentation*”, Available: <https://docs.python.org/3/>
- Flask Documentation, “*Flask Web Framework*”, Available: <https://flask.palletsprojects.com/>
- Scikit-learn Developers, “*Machine Learning in Python*”, Available: <https://scikit-learn.org/>
- K. Ashton, “*That ‘Internet of Things’ Thing*”, RFID Journal, 2009.
- Raj Kamal, “*Internet of Things: Architecture and Design Principles*”, McGraw Hill Education, 2017.
- D. P. Agrawal and Q. A. Zeng, “*Introduction to Wireless and Mobile Systems*”, Cengage Learning, 2015.
- A. Bahga and V. Madisetti, “*Internet of Things: A Hands-On Approach*”, Universities Press, 2014.
- Online Tutorials and Technical Blogs related to IoT, Sensor Integration, and Embedded Systems.